Methodology: Branched Multi-Turn GRPO for Code-Policy Generation

1 Setup and notation

The agent is a language model π_θ that, conditioned on a textual prompt s describing a control task, emits a Python policy. The base is `Qwen2.5-Coder-3B-Instruct` with frozen weights W; θ parameterizes a LoRA adapter giving effective layer weights $W' = W + AB^\top$. Interaction with the environment is mediated by a chat template, so a trajectory is a sequence of role-typed messages.

A task $\mathcal{T}$ specifies environment source code, S scenarios $\{\sigma_1, \ldots, \sigma_S\}$ that vary the seed and hidden parameters, and a per-scenario reward. The scoring oracle runs a candidate policy body in a sandboxed subprocess against each scenario and returns

$$R(\text{code}; \mathcal{T}) \;=\; \sum_{i=1}^{S} r_i(\text{code}; \sigma_i),$$

with r_i the cumulative episode reward (capped at `max_steps`). Validation failures, runtime errors, and timeouts on σ_i score the *error floor* $r_i = -200$, calibrated below any running-but-failing trajectory — otherwise errored rollouts (which accumulate 0) would outrank running-but-poor rollouts that integrate negative distance penalties, and the gradient would reward crashing.

1.1 Multi-turn protocol ($N = 2$)

Each turn the model emits exactly one fenced code block tagged either `policy` or `main`. A `policy` block defines `def policy(state)`; the tool runs it across all scenarios and returns the summed reward as a user-role message. A `main` block is arbitrary scratchpad code with no environment access; the tool returns truncated stdout. The final turn must be `policy`; `main` or untagged final turns are rejected at the floor $-200 \cdot S$. Malformed intermediate turns are likewise floored and surfaced as a textual rejection in the next user message.

The full trajectory is

$$\tau \;=\; \big(s,\; y^{(1)},\; m,\; y^{(2)}\big), \tag{1}$$

where $y^{(1)}, y^{(2)}$ are the two completions and m is the user-role tool message returned after executing $y^{(1)}$. The terminal reward $R(\tau) = R(\text{code in } y^{(2)}; \mathcal{T})$ is the only scalar reward; tool outputs are observations. The rationale for two turns: $y^{(1)}$ either rehearses a draft `policy` (and reads back its actual scenario reward) or runs a `main` scratchpad to print intermediate quantities; $y^{(2)}$ is the scored `policy` we deploy.

2 Background: clipped surrogate without a critic

For a single-turn rollout with completion y of length T, the standard clipped surrogate is

$$\mathcal{L}^{\text{PPO}}(\theta) \;=\; -\,\mathbb{E}_{(s,y)\sim q}\left[\frac{1}{T}\sum_{t=1}^{T} \min\big(\rho_t A_t,\; \text{clip}(\rho_t, 1-\epsilon_{\text{lo}}, 1+\epsilon_{\text{hi}})A_t\big)\right], \tag{2}$$

with $\rho_t(\theta) = \pi_\theta(y_t \mid s, y_{<t})/q(y_t \mid s, y_{<t})$ and q the (frozen) sampling distribution. PPO requires a learned value network for A_t, awkward to fit alongside a LoRA-adapted policy on a single GPU.

Group-relative variants [1] drop the critic in favor of a Monte Carlo per-prompt baseline: sample G completions $\{y^{(g)}\}$, score each, and broadcast a standardized reward to all tokens of $y^{(g)}$,

$$A_t^{(g)} \;=\; \frac{R(y^{(g)}) - \bar{R}}{\widehat{\sigma}_R} \quad \text{(constant in } t\text{)}. \tag{3}$$

We use the *token-summed* variant of the clipped objective: drop the per-row mean, sum the clipped term over all tokens in the global batch, normalize once by the count of valid generated tokens,

$$\mathcal{L}(\theta) \;=\; -\frac{1}{N_{\text{tok}}} \sum_{b,t} \mathbf{1}\{\text{model token}\} \cdot \min\!\big(\rho_t^b A_t^b,\; \text{clip}(\rho_t^b,\, 1-\epsilon_{\text{lo}},\, 1+\epsilon_{\text{hi}}) A_t^b\big), \tag{4}$$

with $N_{\text{tok}} = \sum_{b,t} \mathbf{1}\{\text{model token in } (b,t)\}$.[1] The form is chosen for one structural reason: (4) is *additive over disjoint token subsets* — for any partition of the model-token positions, the loss equals the sum of partial losses with the same N_{tok}. This makes the per-turn decomposition in §4 exact.

3 Branched rollouts and per-turn credit assignment

A direct extension of group-relative training to two-turn trajectories draws G samples per prompt, each containing both turns, and broadcasts a single trajectory-level advantage to every model token in $y^{(1)} \cup y^{(2)}$. This conflates turn-1 and turn-2 credit: a strong turn-1 followed by a weak turn-2 sample receives the same penalty on both turns.

3.1 The $G_1 \times G_2$ tree

We branch at the turn boundary, in the spirit of VinePPO's intra-completion MC branching [3]. For each prompt s, sample G_1 turn-1 variants in one `model.generate` call,

$$y_i^{(1)} \sim q(\cdot \mid s), \qquad i = 1, \ldots, G_1.$$

For each $y_i^{(1)}$, execute the tool to obtain m_i, then sample G_2 continuations in one `generate` call,

$$y_{i,j}^{(2)} \sim q(\cdot \mid s,\, y_i^{(1)},\, m_i), \qquad j = 1, \ldots, G_2.$$

Score each leaf to obtain rewards $\{R_{i,j}\}$. Total generate calls per prompt: $1 + G_1$, versus $2G_1G_2$ for an unbranched $N = 2$ rollout producing the same number of leaves.

3.2 Monte Carlo value estimates and per-turn advantages

Two empirical values, both pure averages over the tree:

$$\widehat{V}_0 \;=\; \frac{1}{G_1 G_2} \sum_{i,j} R_{i,j}, \qquad\qquad \widehat{V}_1^{(i)} \;=\; \frac{1}{G_2} \sum_{j} R_{i,j}. \tag{5}$$

$\widehat{V}_0$ estimates the initial-state value (after the prompt, before any model action); $\widehat{V}_1^{(i)}$ estimates the value after committing to turn-1 variant $y_i^{(1)}$ and observing m_i. With deterministic environments

[1] This matches the DAPO [2] and CISPO recipes; the per-row averaging variants used by the original GRPO formulation, BNPO, and Dr.GRPO instead divide by per-row token counts.

(given the seed) and $\gamma = 1$, both are unbiased MC estimates of the corresponding state values under q.

The per-turn advantages, broadcast to every token of the corresponding turn, are

$$A_i^{(1)} = \widehat{V}_1^{(i)} - \widehat{V}_0, \qquad \text{(over } y_i^{(1)}\text{)}, \tag{6}$$

$$A_{i,j}^{(2)} = R_{i,j} - \widehat{V}_1^{(i)}, \qquad \text{(over } y_{i,j}^{(2)}\text{)}. \tag{7}$$

The turn-2 advantage uses the *conditional* mean given the turn-1 prefix rather than the global per-prompt mean: a poor turn-2 sample drawn from a strong turn-1 is recognized as poor instead of inheriting its prefix's above-average outcome.

Tool-message tokens are masked out via a separate `tool_mask`; their advantage is set to zero. The per-token advantage vector for trajectory (i, j) is

$$A_{i,j} = \big(\underbrace{A_i^{(1)}, \ldots, A_i^{(1)}}_{T_1}, \underbrace{0, \ldots, 0}_{T_m}, \underbrace{A_{i,j}^{(2)}, \ldots, A_{i,j}^{(2)}}_{T_2}\big). \tag{8}$$

3.3 Loss

Plug $A_{i,j}$ into (4), multiply the tool mask into the model-token mask. Standard clip ($\epsilon_{\text{lo}} = 0.2$, asymmetric upper clip), $\beta = 0$, token-level importance sampling. No critic, no GAE, no context truncation across turns.

4 Per-turn loss decomposition

A single forward over the full trajectory $[s \,|\, y^{(1)} \,|\, m \,|\, y^{(2)}]$ materializes an LM-head logits gradient of shape $(B, T_{\text{full}}, |V|)$. With Qwen2.5's $|V| \approx 152{,}000$ this gradient is several gigabytes and OOMs a 24 GB L4 at our trajectory lengths. We exploit additivity of (4) to split the loss into two passes that share parameter gradients but never share activations.

Decomposition. Let $\mathcal{I}_1, \mathcal{I}_2$ index turn-1 and turn-2 model tokens, and $\mathcal{I}_m$ the tool-message tokens. Since the tool mask zeroes the loss on $\mathcal{I}_m$ and (4) sums over positions with a single normalizer,

$$\mathcal{L} = \mathcal{L}_{\mathcal{I}_1} + \mathcal{L}_{\mathcal{I}_2}, \qquad N_{\text{tok}} = |\mathcal{I}_1| + |\mathcal{I}_2| \text{ shared}. \tag{9}$$

Pass 1. Forward $[s \,|\, y^{(1)}]$ with `logits_to_keep`$= T_1^{\max}$, advantages $A_i^{(1)}$ over $\mathcal{I}_1$. Compute $\mathcal{L}_1$, backward (scaled by 1/accum_steps), free.

Pass 2. Forward $[s \,|\, y^{(1)} \,|\, m \,|\, y^{(2)}]$ but pack $[s \,|\, y^{(1)} \,|\, m]$ into the prompt slot and $y^{(2)}$ alone into the completion slot, so the trainer's `logits_to_keep`$= T_2^{\max}$ and the LM-head gradient materializes only over turn-2. Use $A_{i,j}^{(2)}$, compute $\mathcal{L}_2$, backward, free.

By (9) the accumulated parameter gradients sum to the single-pass gradient exactly. Peak memory is max(pass 1, pass 2), since pass-1's graph is freed before pass-2's forward begins.

Restrictions. Equation (9) requires the token-summed, single-normalizer form of (4); per-row averaging variants do not factor across $\mathcal{I}_1 \cup \mathcal{I}_2$. The implementation also needs $\beta = 0$ (a KL term to π_{ref} would require per-segment slicing of reference log-probabilities) and `num_iterations`$= 1$ (so cached `old_per_token_logps` need no re-segmenting across PPO-style iterations).

5 Related work

The closest existing algorithm is *VinePPO* [3], which replaces PPO's learned critic with on-policy MC values obtained by re-branching from chosen intermediate states inside a single chain-of-thought. The granularity is the difference: VinePPO branches at arbitrary intra-completion token positions and uses $\widehat{V}(s_t)$ as a *per-token* baseline that varies within a completion. We branch only at *turn boundaries* and broadcast the resulting advantage as a constant across all tokens of a turn. Within a turn the update is group-relative in the GRPO sense; across turns it is VinePPO-like. Branching depth is fixed at one ($G_1 \times G_2$ tree per prompt), matching the chat-template tool-calling protocol the policy uses at inference time.

A second neighbor is *ArCHer* [7], which uses the same macro-step decomposition (per-turn value, per-token policy update with the turn-level advantage broadcast across action tokens) but estimates the per-turn value with a learned critic. We estimate it by MC over G_2 continuations — the standard variance/bias trade-off of GAE-style versus REINFORCE-style estimates. We take the MC route because $N = 2$ keeps MC variance manageable and avoids fitting a value head alongside the LoRA-adapted policy. A related no-critic alternative is ReBN [6], which replaces GAE with batch normalization of MC returns; the hierarchical mean we use can be viewed as ReBN with the batch partitioned by turn-1 prefix.

The same $G_1 \times G_2$ tree appears in process-reward-model training pipelines such as Math-Shepherd [8] and OmegaPRM [9]: there the per-prefix MC mean labels intermediate steps for a learned reward model. We use it as a policy-gradient baseline rather than as a labelling oracle.

In the broader multi-turn LLM-RL literature, RAGEN's StarPO [4] is the canonical critic-free MC baseline: N trajectories per prompt, flat group baseline over their rewards, single advantage broadcast across all turns. Setting $G_2 = 1$ collapses our scheme to a flat per-prompt baseline of this form. Critic-based turn-level training schemes [5] take the opposite route, fitting GAE on top of a learned value head.

The artifact we train is a *code policy* — `def policy(state)` executed without further LLM calls in the inner control loop. This matches systems like Voyager [10] (frozen-LLM prompting + skill-library accumulation) and FunSearch [11] (LLM as evolutionary mutation operator). Unlike either, we adapt the LLM by gradient-based RL of LoRA weights and use a scratchpad turn between prompt and committed policy.

To our knowledge, the specific combination "turn-boundary branching + critic-free MC baseline + within-turn token broadcast" is not written down in prior LLM-RL work in this exact form, but each ingredient is standard and a reader familiar with VinePPO and ArCHer would not find the specialization surprising. We present it as a clean specialization, not a new algorithm.

6 Summary of contributions

We do not propose a new RL algorithm. The training scheme is VinePPO-style branched MC value estimation with a token-summed clipped surrogate, restricted to $N = 2$ chat-template turns and applied to code-policy generation. Engineering contributions:

1. the per-turn loss decomposition of §4, which exploits additivity of (4) to bound peak LM-head logits memory to max(turn 1, turn 2) instead of their sum, enabling $N = 2$ branched rollouts on a 24 GB GPU;
2. the typed-fence two-turn protocol with a `main` scratchpad turn whose stdout is fed back as a tool message before the final `policy` commit.

References

[1] Z. Shao et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv:2402.03300, 2024.

[2] Q. Yu et al. *DAPO: An Open-Source LLM Reinforcement Learning System at Scale*. arXiv:2503.14476, 2025.

[3] A. Kazemnejad et al. *VinePPO: Unlocking RL Potential For LLM Reasoning Through Refined Credit Assignment*. arXiv:2410.01679, 2024.

[4] Z. Wang et al. *RAGEN: Training Agents by Reinforcing Reasoning*. 2025.

[5] W.-T. Chen et al. *Turn-level training for multi-turn LLM agents*. 2025.

[6] Z. Liu et al. *GEM: Reinforcement Learning for Generalist Models*. 2025.

[7] Y. Zhou et al. *ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL*. ICML, 2024.

[8] P. Wang et al. *Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations*. ACL, 2024.

[9] L. Luo et al. *Improve Mathematical Reasoning in Language Models by Automated Process Supervision*. arXiv:2406.06592, 2024.

[10] G. Wang et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. TMLR, 2024.

[11] B. Romera-Paredes et al. *Mathematical discoveries from program search with large language models*. Nature, 2024.